Lecture 03.06: Loops Using While

📌 Executive Summary

Indefinite iteration using while loops, loop invariants, state mutations, and infinite loop safeguards.

🔬 Theoretical Foundations & Principles

1. Indefinite Iteration Mechanics

A `while` loop repeatedly executes a code suite as long as its controlling predicate evaluates to truth.

```
while condition:
    # Suite executed while condition evaluates to True
    state_mutation()
```

2. Loop Invariants & Termination Proofs

To prove that a while loop terminates, define a monotonic variant function $V : \mathcal{S} \to \mathbb{N}$ such that:

1. $V(s_{k+1}) < V(s_k)$ for each iteration k.
2. The loop terminates when $V(s) \leq 0$.

If no such bounded monotonically decreasing function exists, an **infinite loop** results.

💻 Production Implementation & Code

```
# Monotonic termination example: binary search counter
def countdown_and_sum(n: int) -> int:
    total = 0
    curr = n
    while curr > 0:
        total += curr
        curr -= 1  # Monotonic decrement ensures termination
    return total
```

```
print("Sum of 10 down to 1:", countdown_and_sum(10))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.